

Process Management Part II

Fork():

Fork() is a system call in linux and is the only way to start a child (new) process by the parent process it does not have to an exact copy of it .

If a process uses fork to start, when at the beginning using fork() the an identical copy of the parent is passed as the child.

The calling process (shell) is the one that calls fork() is called the parent process while echo or any other requested process on shell is called a child process.

“The child process has the same program code after creation.” Immediately after fork(): the child process is an exact copy of the parent: Same program code (text segment) Same variables/memory contents at that moment. When executed (or after exec()): The child may run different code (if it calls exec() to run another program, like echo). Variables are always separate from the parent, even before exec()

“After fork(), both parent and child processes start executing at the same point in the code — the line immediately after the fork() call.”

Program counter (PC) = points to the next instruction to run

Both processes resume from the same place, but the return value of fork() tells them whether they are parent or child. Where child has return value of 0 and parent return value of any value greater than zero and -1 means fork failed and child creating failed

Orphan Process

An orphan child exists when a parent process ends before the child exits its process

SysVinit or the latest system: it's the init process and every running process on the OS is a child or a grand child of this process. When a child process gets orphanized the OS passes this process into the system and it handles managing and cleaning this process, the OS assigns A PPID of 1 to the orphan process.

The system process is the first thing that loads from the OS after the kernel is loaded. When you write pstree (process tree) it will show you the running processes which starts at the top of the tree with systemd (the init process)

Fork Bomb:

A fork bomb is a program that keeps calling `fork()` in an infinite loop. This means it keeps creating child processes forever as fast as the system allows. Each child also continues the loop and forks more children.

What happens then?: The process table becomes full, The CPU gets overloaded and the system runs out of memory, The computer becomes very slow or completely freezes.

Process Replacement – Chaining:

When you `fork()` a process a new fresh copy starts of the parent exact copy, when you then call `exec(program)` the `exec()` changes the body of the program and then the process changes from x to y with the same PID but cleared stack, heap, and a new text BSS and Data segments of the program passed to the `exec()`. All of this does not affect the parent process that did the `fork()`

Example: the bash (parent process) calls the `fork()` to fork a process that we will call X.

This X then calls the command `exec(firefox)` now our process X is changed into Y and has a cleared stack and heap memory, a new text (code of firefox) new BSS for uninitialized variables and Data segment of fire fox. The bash is untouched.